The Swirl Activation Function

L. David Aites oneofthemanydavids@gmail.com
Iowa City, IA 52246, USA

Editor: Editor

Abstract

Artificial neural networks have been receiving continually more attention as more uses are found for them. Artificial neural networks require activation functions but currently available functions are known to have flaws. They are also not always applicable to every type of model. Here we present the Swirl activation function and some modifications to it which are meant to help mitigate these issues.

1 Introduction

Artificial neural networks (ANNs), especially deep learning architectures, are finding continually more applications as well as more attention. At their fundamental level artificial neural networks are modeled after biological neural networks; they are built out of layers of neurons that connect to each other and allow the gathering and processing of information. For artificial neural networks it is common to use activation functions to add non-linearity between the layers of artificial neurons (Dubey et al. 2022; LeCun et al. 2015). Different activation functions have different mathematical traits and also may perform quite differently when applied to different architectures, tasks, and data sets (Lederer 2021). Some have known flaws that cause various problems for ANNs; a common example being the Dying ReLU Problem pertaining to the rectified linear unit (ReLU) function. While the ReLU activation function is commonly used due to its potentially high accuracy it is also prone to collapsing into an inaccurate state especially as the number of layers in a neural network increases (Lu et al. 2020). Meanwhile other activation functions have limits in what they can accomplish as they are strictly bounded (Lederer 2021).

We present here the Swirl activation function as well as modifications to it which endeavor to mitigate some of these issues. Swirl is novel to the best of our knowledge. The basic idea of the Swirl function is the equation and its derivative as such:

$$\sigma(x) = \frac{x}{e^{|x|-1}} \qquad \sigma'(x) = -e^{1-|x|}(|x| - 1)$$

which can then be further modified. In particular parameters may be added to make the function adaptable as such:

$$\sigma(x) = \frac{\alpha x}{e^{|\beta x|-1}} \qquad \sigma'(x) = -\alpha e^{1-|\beta||x|}(|\beta||x| - 1)$$

These parameters may either be set ahead of time or may be used as learnable parameters. This function is bounded as [-1, 1] without any learnable parameters. With learnable

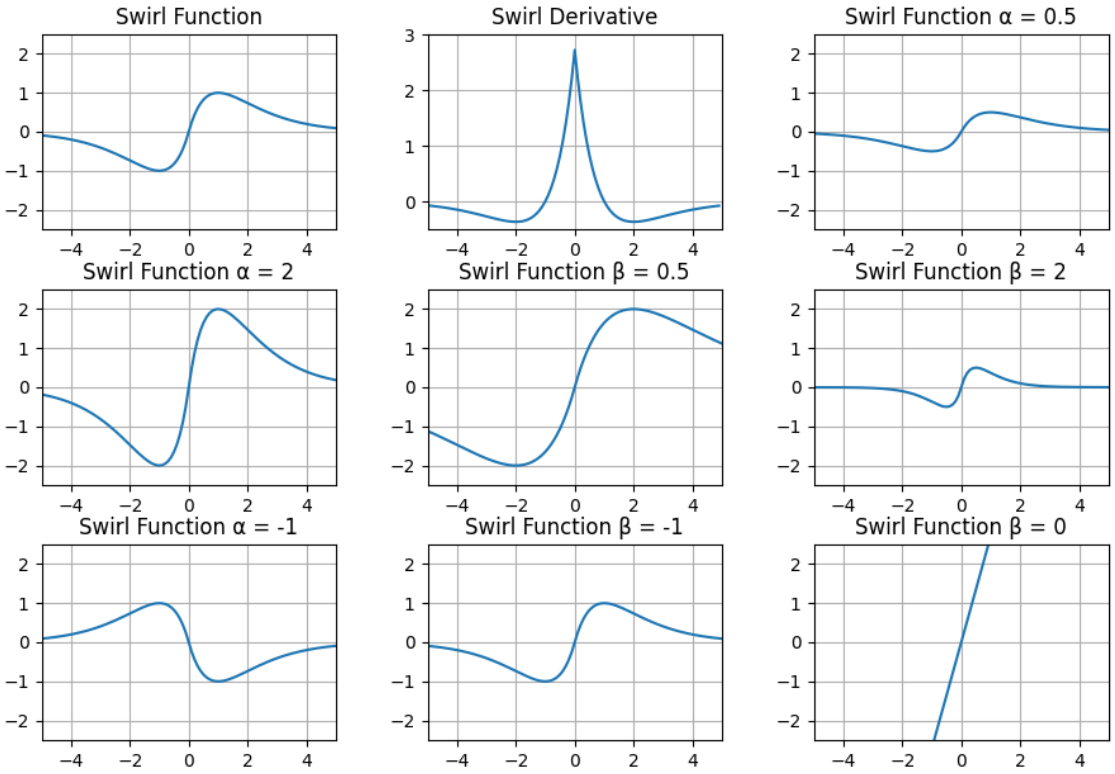

Figure 1: Swirl Function

parameters this is bounded as $[\frac{-|\alpha|}{|\beta|}, \frac{|\alpha|}{|\beta|}]$ when $\beta \neq 0$. For $\beta = 0$ the function becomes linear and the bounds are infinite.

Further modifications as well as their performance will be discussed.

2 Assumptions

The creation of the swirl function was based on some assumptions made about what makes a good activation function. First is the accuracy of the models that include it. It is obvious that the most desirable attribute of any tool is how well it does the job it is expected to do. It is also assumed that a high quality activation function will have broad potential applications. It should perform well on as many types of architectures or data sets as possible.

It is also assumed that good activation functions should be adaptable. A common issue among activation functions is that they have strict bounds. Though they are common the sigmoid and hyperbolic tangent (tanh) functions are strictly bounded; the sigmoid function has a range of (0, 1) while tanh has a range of (-1, 1). Meanwhile ReLU is strictly non-negative (Lederer 2021). While the Gaussian Error Linear Unit (GELU) has shown recent promise it still has a lower bound of approximately -0.17 (Hendrycks and Gimpel 2016).

Lastly it is assumed that a good activation function will be predictable. If a model is trained multiple times the final accuracies of each training run should be fairly close. It is assumed that a wide range of accuracies is undesirable as it suggests that the model is un-

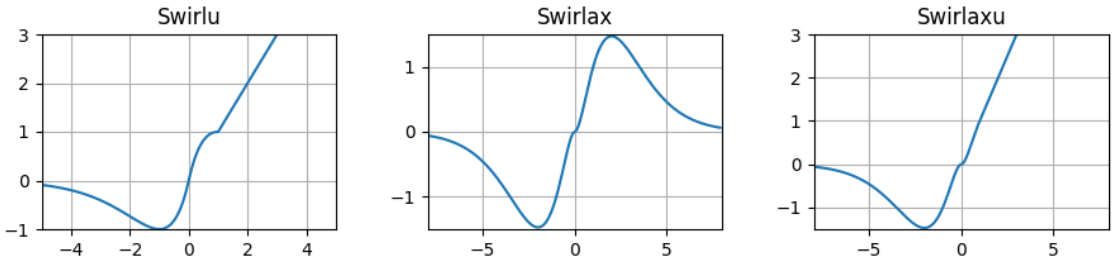

Figure 2: Swirl Modifications

stable or fails to properly converge on a consistent basis. It is obvious that this is especially important when dealing with models that have a long training time; if multiple training runs must be performed this can become computationally expensive to an undesirable level.

3 Modifications

Modifications of the Swirl function are included here that seek to expand the possible applications of the function. Of particular note is modifying them to behave similarly to ReLU beyond the maximum of the function. Beyond the above illustrated basic function the following functions will also be discussed. All functions may also be enhanced with learnable parameters; they will be included beside the base function.

3.1 Swirlu

$$\sigma(x) = \left\{ \begin{array}{ll} x & \text{for } x \geq 1 \\ \frac{x}{e^{|x|-1}} & \text{for } x < 1 \end{array} \right\} \quad \sigma(x) = \left\{ \begin{array}{ll} \alpha x & \text{for } x \geq \frac{1}{|\beta|} \\ \frac{\alpha x}{e^{|\beta x|-1}} & \text{for } x < \frac{1}{|\beta|} \end{array} \right\}$$

3.2 Swirlax

$$\sigma(x) = \frac{|x|x}{e^{|x|-1}} \quad \sigma(x) = \frac{\alpha|x|x}{e^{|\beta x|-1}}$$

3.3 Swirlaxu

$$\sigma(x) = \left\{ \begin{array}{ll} x & \text{for } x \geq 1 \\ \frac{|x|x}{e^{|x|-1}} & \text{for } x < 1 \end{array} \right\} \quad \sigma(x) = \left\{ \begin{array}{ll} \frac{\alpha x}{|\beta|} & \text{for } x \geq \frac{1}{|\beta|} \\ \frac{\alpha|x|x}{e^{|\beta x|-1}} & \text{for } x < \frac{1}{|\beta|} \end{array} \right\}$$

4 Testing

As it is assumed that a good activation function will be applicable to different tasks and architectures a selection of models were tested. This includes basic mutlilayer perceptrons (MLPs) (Almeida 1996), convolutional neural networks (CNNs) (LeCun et al. 1989), and graph convolutional neural networks (GCNs) (Kipf and Welling 2017). These will be applied to classification tasks. Similarly a variety of data sets were chosen and tested. The varieties of the Swirl function are tested using the MNIST (LeCun et al. 2010), Cora (McCallum et al. 2000), CiteSeer (Giles et al. 1998), PubMed (Yang et al. 2016), CIFAR-10 (Krizhevsky 2009), and Sonar (Gorman and Sejnowski 1988) data sets. Swirl was tested both with and

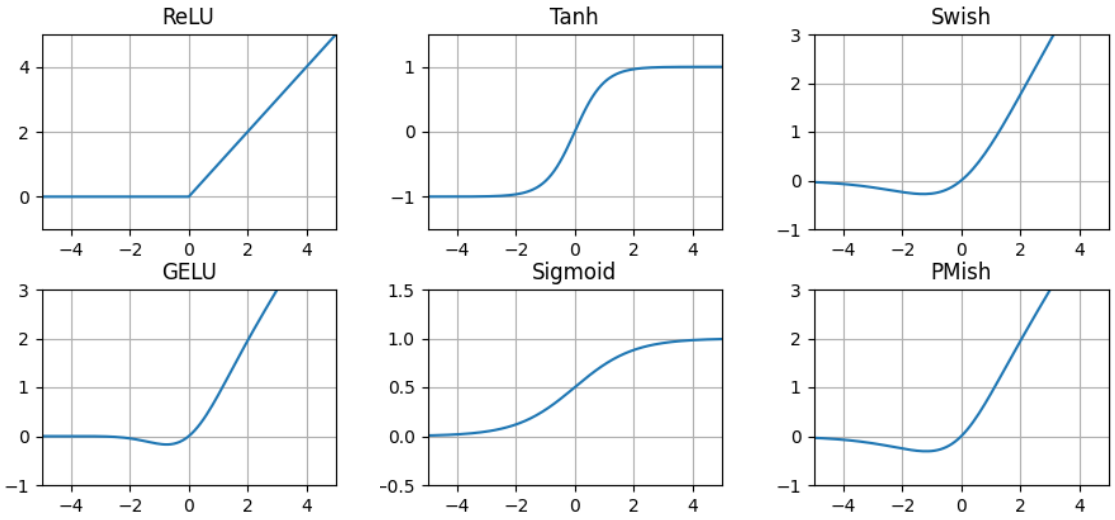

Figure 3: Other Functions (Lederer 2021, Hendrycks and Gimpel 2016, Maas et al. 2013, Ramachandran et al. 2018, Pulakurthi et al. 2024)

without learnable parameters; if the model uses learnable parameters it is marked with L in the charts.

The models chosen were also tested using the Tanh, Sigmoid, ReLU, (Lederer 2021), GELU (Hendrycks and Gimpel 2016), Leaky ReLU (Maas et al. 2013), Swish (Ramachandran et al. 2018), and PMish (Pulakurthi et al. 2024) activation functions. For the purposes of fairness all functions were tested utilizing the same models, optimizers, number of epochs, and learning rates unless a particular model performed extremely poorly or collapsed. In these cases similar values were sought that would allow it to perform well whenever possible; it is noted where this is done. Whenever possible values were chosen that allowed all models to converge after training if they were capable of converging. Some activation functions it was found perform poorly on certain models or data sets. Specifics of the models are included in the appendix.

$$ReLU : \sigma(x) = \left\{ \begin{array}{ll} x & \text{for } x \geq 0 \\ 0 & \text{for } x < 0 \end{array} \right\}$$

$$Tanh : \sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$Sigmoid : \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$LeakyReLU : \sigma(x) = \left\{ \begin{array}{ll} x & \text{for } x \geq 0 \\ \alpha x & \text{for } x < 0 \end{array} \right\} \text{where } \alpha \leq 1$$

$$Swish : \sigma(x) = \frac{x}{1 + e^{-x}}$$

$$GELU : \sigma(x) = \frac{x}{2}(1 + erf(\frac{x}{\sqrt{2}}))$$

$$PMish : \sigma(x) = xtanh(\frac{ln(1 + e^{\beta x})}{\beta}), \beta > 0$$

5 Evaluation

Our measure of the performance of the Swirl function is based on multiple metrics gathered while testing. First is the average percent accuracy level achieved from training a given model with the provided activation function ten times. Second is the minimum accuracy from those runs. Third is the maximum accuracy achieved. It is of course possible to train a model multiple times and select the best result. Lastly is the variation in accuracies; lower being better as it is assumed it indicates consistency in where the model converges. The variation is calculated as the larger difference of the minimum or maximum from the average. These are then combined into the average performance across all tested models and data sets.

It is indicated which functions performed best on the average, maximum, and variation for each data set and model. The included charts include information on which did best on each of these metrics. All model performance metrics are included in the appendix.

6 Basic and Convolutional Neural Networks - MNIST

The MNIST data set is a collection of hand-written numbers separated into a 60,000 example training set and a 10,000 example test set. Each example is a 28x28 grayscale image consisting of one handwritten number. The task for this data set is to correctly identify which number is in the image (LeCun et al. 2010). This data set was used to test a single layer neural network, two and three layer multilayer percpetrons, and a two convolution layer model.

Model	Function	Avg	Min	Max	+/-
Single Layer	Swirlax L	**0.9289**	0.9284	**0.9295**	0.00059
Single Layer	Swirlaxu	0.9197	0.9195	0.9201	**0.0004**
Single Layer	Swirlaxu L	**0.9289**	**0.9285**	**0.9295**	0.00062
2 Layer MLP	Swirl L	**0.9858**	**0.9853**	**0.9864**	0.00062
2 Layer MLP	GELU	0.9801	0.9796	0.9807	**0.00058**
3 Layer MLP	Swirl L	**0.9853**	**0.9845**	0.9859	0.00081
3 Layer MLP	Swirlax L	0.9852	0.9844	**0.9862**	0.001
3 Layer MLP	Swirlu	0.9844	0.9839	0.9848	**0.00051**

Figure 4: MNIST Regular Network Best Functions

Model	Function	Avg	Min	Max	+/-
Convolution	PMish	**0.9921**	0.9912	0.9927	0.00089
Convolution	ReLU	0.963	0.8944	**0.9929**	0.06863
Convolution	Tanh	0.991	0.9904	0.9915	**0.00058**

Figure 5: MNIST Convolution Best Functions

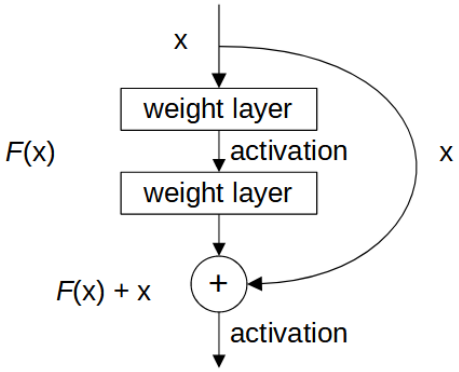

Figure 6: Residual Learning Building Block (He et al. 2016)

7 Deep Residual Networks - CIFAR-10

The CIFAR-10 data set is a set of 60,000 32x32 color images split into 50,000 training images and 10,000 test images. They are equally distributed between 10 classes with no overlap. These are images of airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Note that the trucks are only large trucks while the automobiles are smaller motor vehicles; both excluding small trucks such as pickup trucks. The task here is to classify each image into which category it is a member of (Krizhevsky 2009).

The ResNet-18 model was trained on this data set. ResNet models overall are deep learning models constructed out of residual mappings between stacked layers. This is done by passing the input of one stack of layers to the last layer and adding it to the output of the given stack. This is done to mitigate the degradation problem that can exist in deeper models; this can ensure that a deeper model's training error is limited to that of the shallower one. These can be stacked deeply; sometimes to hundreds of layers. Here we

Function	Avg	Min	Max	+/-
PMish	**0.9106**	0.9028	0.9155	0.0078
GELU	0.9101	**0.9059**	**0.9171**	0.007
Swirlu L	0.9001	0.8965	0.9029	**0.0036**

Figure 7: CIFAR-10 ResNet-18 Best Functions

Model	Function	Avg	Min	Max	+/-
Deep	Swirl	**0.8795**	0.8434	0.9036	0.0361
Deep	Swirlu L	0.8711	**0.8554**	**0.9157**	0.0446
Deep	Tanh	0.7759	0.759	0.7952	**0.0193**
Shallow	Swirlu L	**0.8627**	0.8313	**0.9277**	0.0651
Shallow	Swirlu	0.8446	0.8193	0.8675	**0.0253**

Figure 8: Sonar Best Functions

chose the 18 layer model. The original implementation of ResNet only includes the ReLU activation function (He et al. 2016). We tested it with all of the chosen activation functions.

8 Small Data Sets - Sonar

The basic sonar data set used here is a simple, small data set of 208 samples of sonar relating to 97 rocks and 111 metal cylinders. The goal is to classify each data point as one or the other (Gorman and Sejnowski 1988). We split this into 125 training samples and 83 test samples. It was tested on a shallow MLP with two hidden layers and a deeper model with seven hidden layers.

9 Graph Convolutional Neural Networks - Cora, Citeseer, and PubMed

The Cora, Citeseer, and PubMed data sets are each citation network data sets of varying sizes. In each data set each node is a specific paper and each edge indicates a citation exists between the papers. All edges are bi-directional regardless of which paper cites the other. All papers are classified into a number of classes depending on the data set. The task here is to predict which class a paper belongs to (Yang et al. 2016, McCallum et al. 2000, Giles et al. 1998, Sen et al. 2008).

We tested these data sets on a two layer graph convolutional neural network (GCN) model described as such:

$$\mathcal{Z} = f(\mathcal{X}, \mathcal{A}) = softmax(\hat{\mathcal{A}}\sigma(\hat{\mathcal{A}}\mathcal{X}\Theta^{(0)})\Theta^{(1)})$$

where $\hat{\mathcal{A}} = \tilde{\mathcal{D}}^{-1/2}\tilde{\mathcal{A}}\tilde{\mathcal{D}}^{-1/2}$. Here $\mathcal{D}$ is the degree matrix of the given graph, $\mathcal{A}$ is the adjacency matrix, $\mathcal{X}$ is the graph features, Θ is a weight matrix, and σ is an activation function. $\tilde{\mathcal{D}}$ and $\tilde{\mathcal{A}}$ refer to degree and adjacency matrices that include self-references for each node (Kipf and Welling 2017). This utilizes the fixed public split of training and test data sets as described by Yang, Cohen, and Salakhutdinov (Yang et al. 2016).

We also tested these on informed graph convolutional neural networks (IGCN) with a single layer neural network as well as a three hidden layer neural network. Informed graph convolution performs all graph convolution as a single step before any training is performed. New feature vectors n are calculated as such:

$$n_i = h_i || \sum_{j=1, j\neq i}^{x} \hat{\mathcal{A}}_{ij}^{1} h_j || \sum_{j=1, j\neq i}^{x} \hat{\mathcal{A}}_{ij}^{2} h_j || ... \sum_{j=1, j\neq i}^{x} \hat{\mathcal{A}}_{ij}^{K} h_j$$

Data Set	Function	Avg	Min	Max	+/-
Cora	Swirlaxu L	**0.8136**	**0.81**	0.818	0.0044
Cora	Sigmoid	0.8115	0.808	**0.819**	0.0075
Cora	Tanh	0.8071	0.806	0.808	**0.0011**
Citeseer	Sigmoid	**0.6938**	0.681	**0.708**	0.0142
Citeseer	Swirl	0.6807	0.679	0.682	**0.0017**
PubMed	GELU	**0.7897**	0.783	**0.795**	0.0067
PubMed	Swish	0.7866	0.777	**0.795**	0.0096
PubMed	Swirl L	0.7856	0.785	0.787	**0.0014**

Figure 9: Double Layer GCN Best Functions

where h is the original feature vectors, x is the count of feature vectors, K is the number of hops through the graph desired, and $||$ is used for concatenation. This concatenates the original, unmodified node features with increasingly large neighborhoods of features calculated that exclude the original nodes using the normalized adjacency matrices described previously (Aites and Stewart 2024). This is more simply described using $S_N = \hat{\mathcal{A}}^N$ where for each S_N the diagonal of the matrix is set to 0. From here we may notate the resulting feature vectors as $\hat{\mathcal{X}} = \mathcal{X}||\Xi_{N=1}^{K} S_N \mathcal{X}$ using $||$ for feature vector-wise concatenation and Ξ for repeated concatenation. The end result will be a new set of feature vectors per node as previously described.

IGCN utilizes a 60/40 split for the training and testing sets.

10 Results

Overall the best performing function among those tested was Swirlu. Baseline Swirl and Swirlu performed well in general either better than or comparable to other chosen activation functions. Swirlax and Swirlaxu varieties failed to perform as well overall but were the best tested function on some of the given tasks.

ReLU was shown to be unstable. It has a high variation far worse than other functions. This can be mitigated by using the Leaky ReLU variation but this still has some of the worst variation among the activation functions tested.

Data Set	K	Function	Avg	Min	Max	+/-
Cora	3	Swirlax L	**0.8771**	0.8735	**0.88**	0.0036
Cora	3	Tanh	0.8712	0.8698	0.8726	**0.00139**
Citeseer	3	Sigmoid	**0.7633**	**0.7618**	0.7648	**0.00158**
Citeseer	3	Swirlu	0.7612	0.7581	**0.7663**	0.00511
Citeseer	3	GELU	0.7543	0.7536	0.7558	**0.0015**
PubMed	2	Swirlax L	**0.8772**	**0.8756**	**0.8799**	0.00278
PubMed	2	PMish	0.8391	0.8388	0.8392	**0.00027**

Figure 10: Single Layer IGCN Best Functions (Best K Value Chosen)

Data Set	K	Function	Avg	Min	Max	+/-
Cora	5	Swirlu L	**0.8706**	0.8652	**0.8772**	0.00656
Cora	5	Swirl	0.8536	0.8504	0.8569	**0.00323**
Citeseer	2	Swirlu	**0.7596**	0.7551	0.7633	0.00451
Citeseer	2	Swirlaxu	0.7554	0.7438	**0.7671**	0.01172
Citeseer	2	Swirlaxu L	0.7594	0.7528	**0.7671**	0.00774
Citeseer	2	Swirlu L	0.7583	0.7551	0.7611	**0.00323**
PubMed	2	Swirl	**0.8947**	**0.8926**	**0.8964**	0.00212
PubMed	2	Swirl L	0.8935	**0.8926**	0.8945	**0.00099**

Figure 11: Four Layer IGCN Best Functions (Best K Value Chosen)

Function	Avg	Min	Max	+/-
Swirl	0.8519	**0.8462**	0.8572	0.0065
Swirl L	**0.8533**	0.8339	**0.8625**	0.0197
Swirlu	**0.8592**	**0.8536**	**0.8652**	**0.0064**
Swirlu L	**0.8617**	**0.8567**	**0.8706**	0.0091
Swirlax	0.8294	0.818	0.8354	0.0118
Swirlax L	0.8366	0.8212	0.8427	0.0155
Swirlaxu	0.8341	0.8188	0.8421	0.0156
Swirlaxu L	0.827	0.8126	0.8413	0.0192
ReLU	0.7779	0.6741	0.8381	0.1097
Leaky ReLU	0.8483	0.8307	0.856	0.0179
Tanh	0.8479	0.8423	0.8531	**0.006**
Swish	0.8322	0.8217	0.8397	0.0107
GELU	0.8367	0.8311	0.8413	**0.0059**
Sigmoid	0.6996	0.6913	0.718	0.0192
PMish	0.8408	0.8238	0.8543	0.0194

Figure 12: Overall Performance

Tanh and GELU were the two most stable of the functions tested but comparable stability could be found by using Swirl or Swirlu. Meanwhile Swirlu allowed more accuracy than these functions.

Sigmoid failed at some tasks. Overall it was the tested function that had the most issues. Swirlax and Swirlaxu sometimes also failed at various tasks; testing showed that Swirlaxu in particular was especially prone to accuracy collapsing during training.

The performance of Swish and PMish were middling; while their performance was good overall they lacked the performance of other functions.

Interestingly adding learnable parameters to the swirl varieties can improve their performance but makes them less stable overall.

Data Set	Model	Hidden Layers	Epochs
MNIST	Single Layer	None	450
MNIST	One Hidden Layer	1568	2000
MNIST	Two Hidden Layer	1568/768	2500
MNIST	Convolution	Conv/Conv/192	50
Sonar	Shallow	30/30	250
Sonar	Deep	240/120/60/30/60/120/240	500
CIFAR-10	ResNet-18	Residual and Convolution	120
Cora, Citeseer, Pubmed	Two Layer GCN	GCN/GCN	400
Cora, Citeseer, Pubmed	IGCN	None	600
Cora, Citeseer, Pubmed	IGCN with Layers	128/64/128	700

Figure 13: Models

	Shallow				Deep			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	**0.8614**	**0.8434**	**0.8916**	**0.0301**	**0.8795**	**0.8434**	**0.9036**	**0.0361**
Swirl L	**0.8482**	0.8072	**0.8916**	0.0434	**0.8783**	0.8313	**0.9036**	0.047
Swirlu	0.8446	**0.8193**	0.8675	**0.0253**	0.8639	0.8313	**0.9036**	**0.0398**
Swirlu L	**0.8627**	**0.8313**	**0.9277**	0.0651	**0.8711**	**0.8554**	**0.9157**	0.0446
Swirlax	0.8289	0.7952	0.8554	**0.0337**	0.5253	0.4578	0.5422	0.0675
Swirlax L	0.8012	0.747	0.8313	0.0542	0.5337	0.4578	0.5422	0.0759
Swirlaxu	0.8108	0.7349	0.8675	0.0759	0.5337	0.4578	0.5422	0.0759
Swirlaxu L	0.8157	0.7831	0.8554	0.0398	0.5337	0.4578	0.5422	0.0759
ReLU	0.8108	0.7229	0.8554	0.088	0.5855	0.4578	0.8072	0.2217
Leaky ReLU	0.8217	0.7711	0.8434	0.0506	0.7651	0.6988	0.7952	0.0663
Tanh	0.7807	0.7349	0.8193	0.0458	0.7759	0.759	0.7952	**0.0193**
Swish	0.8012	0.7349	0.8434	0.0663	0.5422	0.5422	0.5422	0
GELU	0.8169	0.7831	0.8434	**0.0337**	0.5422	0.5422	0.5422	0
Sigmoid	0.5422	0.5422	0.5422	0	0.5422	0.5422	0.5422	0
PMish	0.8084	0.7108	0.8434	0.0976	0.6337	0.5422	0.759	0.1253

Figure 14: All Sonar Results

11 Appendix

The results of all runs are included here. This does not include the individual training results but rather the avarage, minimum, maximum, and spread for each model and data set combination for each activation function.

11.1 Model Details

For all models the optimizer is Adam (Kingma and Ba 2015) and the loss function is cross entropy loss (Zhang and Sabuncu 2018). Learning rates vary and are generally reduced during training. For more information on this consult the publicly posted code. This project was done using PyTorch (Ansel et al. 2024)

	Single Layer				Single Hidden Layer			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	0.9015	0.9011	0.902	0.00049	**0.982**	**0.9811**	**0.9828**	0.00092
Swirl L	**0.9279**	**0.9275**	**0.9284**	0.00052	**0.9858**	**0.9853**	**0.9864**	**0.00062**
Swirlu	0.9193	0.9188	0.9197	0.00051	0.9803	0.9794	0.9814	0.00108
Swirlu L	0.9238	0.9233	0.9248	0.00103	0.9812	0.9803	0.9821	0.00091
Swirlax	0.8949	0.8942	0.8956	0.00073	0.9755	0.9747	0.9762	0.00077
Swirlax L	**0.9289**	**0.9284**	**0.9295**	0.00059	**0.984**	**0.9834**	**0.9845**	**0.00064**
Swirlaxu	0.9197	0.9195	0.9201	**0.0004**	0.9706	0.9693	0.972	0.0014
Swirlaxu L	**0.9289**	**0.9285**	**0.9295**	0.00062	0.973	0.9722	0.9737	0.00076
ReLU	0.8619	0.7307	0.9209	0.13117	0.8921	0.776	0.9813	0.11611
Leaky ReLU	0.9112	0.8385	0.9203	0.07265	0.981	0.9806	0.9818	0.0008
Tanh	0.9084	0.9076	0.9092	0.0008	0.9763	0.9756	0.9769	0.00071
Swish	0.9205	0.92	0.9208	0.00047	0.977	0.9765	0.9777	0.00067
GELU	0.9204	0.92	0.9208	**0.00042**	0.9801	0.9796	0.9807	**0.00058**
Sigmoid	0.897	0.8963	0.8977	0.00071	0.8733	0.8724	0.8737	0.00088
PMish	0.9203	0.9199	0.9207	**0.00042**	0.9784	0.9776	0.9794	0.00101
	Double Hidden Layer				Convolution			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	**0.9845**	**0.9839**	0.9849	**0.00058**	0.9898	0.9878	0.992	0.00218
Swirl L	**0.9853**	**0.9845**	**0.9859**	0.00081	0.9621	0.8043	0.9916	0.15777
Swirlu	0.9844	**0.9839**	0.9848	**0.00051**	0.9907	0.9895	0.9916	0.00122
Swirlu L	0.9844	0.9838	**0.9851**	**0.00071**	0.991	0.9895	0.9925	0.00151
Swirlax	0.9804	0.9797	0.9813	0.00095	0.9908	0.9902	0.9915	0.0007
Swirlax L	**0.9852**	**0.9844**	**0.9862**	0.001	0.9908	0.9897	0.9918	0.00107
Swirlaxu	0.9782	0.9685	0.9798	0.00969	0.9911	0.9904	0.9923	0.00121
Swirlaxu L	0.9812	0.98	0.9817	0.00122	0.9907	0.9902	0.9918	0.00111
ReLU	0.965	0.8879	0.9843	0.07705	0.963	0.8944	**0.9929**	0.06863
Leaky ReLU	0.9833	0.9825	0.985	0.00175	**0.9921**	**0.9914**	**0.9927**	**0.00065**
Tanh	0.9823	0.9818	0.9831	0.00076	0.991	0.9904	0.9915	**0.00058**
Swish	0.9802	0.9795	0.9809	0.00072	**0.992**	**0.9913**	0.9925	**0.00068**
GELU	0.9821	0.9812	0.9833	0.00119	**0.992**	**0.9912**	**0.9927**	0.00076
Sigmoid	0.8329	0.788	0.9602	0.12735	0.9816	0.9796	0.9846	0.00304
PMish	0.9813	0.9804	0.9818	0.00089	**0.9921**	**0.9912**	**0.9927**	0.00089

Figure 15: All MNIST Results

The basic IGCN model utilized ReLU with a lower learning rate on the PubMed data set. The model utilized lower learning rates for the Swirlaxu Learnable function for all data sets.

The IGCN with layers model utilized ReLU and Sigmoid with reduced learning rates.

The ResNet-18 model used lower learning rates for the Swirlu Learnable, Swirlu 2 Learnable, PMish, and Swirlaxu Learnable functions.

Specific details are available in the publicly posted code.

	ResNet-18				GCN Cora			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	0.8563	0.8498	0.8612	0.0065	0.8012	0.798	0.805	0.0038
Swirl L	0.8077	0.7706	0.828	0.0371	0.8021	0.8	0.804	**0.0021**
Swirlu	0.8961	0.8912	0.9	**0.0049**	0.8011	0.798	0.806	0.0049
Swirlu L	0.9001	0.8965	0.9029	**0.0036**	0.8014	0.798	0.804	0.0034
Swirlax	0.888	0.8831	0.8925	**0.0049**	0.8127	**0.81**	0.814	0.0027
Swirlax L	0.8943	0.8783	0.903	0.016	**0.813**	0.803	0.816	0.01
Swirlaxu	0.8999	0.8943	0.9034	0.0056	**0.8131**	**0.81**	0.816	0.0031
Swirlaxu L	0.9056	0.9018	0.9114	0.0059	**0.8136**	**0.81**	**0.818**	0.0044
ReLU	0.9061	0.8984	0.9115	0.0077	0.8071	0.805	0.809	0.0021
Leaky ReLU	**0.9076**	0.9004	**0.9135**	0.0072	0.8063	0.805	0.809	0.0027
Tanh	0.8961	0.8901	0.9002	0.006	0.8071	0.806	0.808	**0.0011**
Swish	0.906	**0.902**	0.9112	0.0052	0.8119	0.809	**0.817**	0.0051
GELU	**0.9101**	**0.9059**	**0.9171**	0.007	**0.813**	0.809	0.815	0.004
Sigmoid	0.8708	0.8598	0.8769	0.011	0.8115	0.808	**0.819**	0.0075
PMish	**0.9106**	**0.9028**	**0.9155**	0.0078	0.8112	**0.81**	0.812	**0.0012**

Figure 16: All ResNet-18 and Cora GCN Results

	GCN Citeseer				GCN PubMed			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	0.6807	0.679	0.682	**0.0017**	0.7865	**0.785**	0.788	**0.0015**
Swirl L	0.6806	0.678	0.683	0.0026	0.7856	**0.785**	0.787	**0.0014**
Swirlu	0.6813	0.678	0.686	0.0047	0.7861	**0.785**	0.788	0.0019
Swirlu L	0.6805	0.679	0.683	**0.0025**	0.7856	0.784	0.787	**0.0016**
Swirlax	0.6836	0.67	0.69	0.0136	0.783	0.773	0.792	0.01
Swirlax L	0.6737	0.639	0.686	0.0347	0.7791	0.764	0.787	0.0151
Swirlaxu	0.6812	0.662	**0.691**	0.0192	0.7855	0.781	0.792	0.0065
Swirlaxu L	0.6768	0.663	0.686	0.0138	0.778	0.768	0.787	0.01
ReLU	**0.6858**	**0.683**	0.69	0.0042	**0.7889**	**0.785**	0.793	0.0041
Leaky ReLU	**0.6855**	**0.683**	0.688	**0.0025**	0.7877	0.779	**0.794**	0.0087
Tanh	0.6852	**0.684**	0.689	0.0038	**0.7883**	**0.786**	0.79	0.0023
Swish	0.6832	0.67	0.69	0.0132	0.7866	0.777	**0.795**	0.0096
GELU	0.6826	0.674	0.69	0.0086	**0.7897**	0.783	**0.795**	0.0067
Sigmoid	**0.6938**	0.681	**0.708**	0.0142	0.7782	0.758	0.793	0.0202
PMish	0.6848	0.68	**0.693**	0.0082	0.7865	0.781	0.792	0.0055

Figure 17: All GCN Citeseer and PubMed Results

11.2 Hardware

All training was done on an NVIDIA GeForce RTX 5070 Ti with 16 GB of RAM on a computer with an AMD Ryzen 7 9800X3D processor and 128 GB of RAM.

11.3 Source Code

The source code will be made avalable at https://github.com/DavidAites/swirl.

	Simple IGCN Cora K=3				Simple IGCN Citeseer K=3			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	0.8415	0.8366	0.8449	0.00489	0.72	0.7175	0.722	0.00248
Swirl L	0.8723	0.8707	0.8744	0.00212	0.7554	0.7491	0.7603	0.00639
Swirlu	**0.8765**	**0.8744**	**0.8781**	**0.00203**	**0.7612**	**0.7581**	**0.7663**	0.00511
Swirlu L	0.8734	0.8698	**0.8781**	0.00471	**0.7615**	**0.7588**	**0.7641**	0.00263
Swirlax	0.851	0.8476	0.856	0.00499	0.7301	0.7273	0.737	0.00691
Swirlax L	**0.8771**	**0.8735**	**0.88**	0.0036	0.7518	0.7483	0.7536	0.00346
Swirlaxu	**0.8763**	**0.8735**	**0.879**	0.00277	0.7569	0.7558	0.7596	0.00263
Swirlaxu L	0.8737	0.8689	0.8763	0.0048	0.7541	0.7513	0.7558	0.00278
ReLU	0.8704	0.8689	0.8726	0.00222	0.756	0.7536	0.7581	0.0024
Leaky ReLU	0.87	0.8652	0.8735	0.0048	0.7557	0.7536	0.7573	0.0021
Tanh	0.8712	0.8698	0.8726	**0.00139**	0.7598	0.7566	0.7618	0.00323
Swish	0.8693	0.867	0.8717	0.0024	0.7545	0.7528	0.7566	**0.00203**
GELU	0.8701	0.868	0.8726	0.00249	0.7543	0.7536	0.7558	**0.0015**
Sigmoid	0.8727	0.8717	0.8744	**0.00175**	**0.7633**	**0.7618**	**0.7648**	**0.00158**
PMish	0.8693	0.867	0.8707	0.00222	0.7545	0.7528	0.7566	0.0021
	Simple IGCN PubMed K=2				Deep IGCN Cora K=5			
Swirl	0.8515	0.8509	0.8524	0.00094	0.8536	0.8504	0.8569	**0.00323**
Swirl L	**0.8681**	**0.8678**	**0.8689**	0.00082	0.8551	0.8504	0.8596	0.00471
Swirlu	0.8484	0.8476	0.8492	0.00086	**0.8694**	**0.8652**	0.8726	**0.00425**
Swirlu L	**0.857**	**0.8563**	**0.8576**	0.00066	**0.8706**	**0.8652**	**0.8772**	0.00656
Swirlax	0.8366	0.834	0.8377	0.00256	0.8596	0.8421	0.8652	0.01745
Swirlax L	**0.8772**	**0.8756**	**0.8799**	0.00278	0.8613	0.8458	0.868	0.01551
Swirlaxu	0.837	0.8345	0.838	0.00251	0.8665	0.8606	0.8744	0.00794
Swirlaxu L	0.7054	0.6442	0.8286	0.12323	0.8678	0.8615	**0.8763**	0.00849
ReLU	0.8377	0.8366	0.8385	0.00115	0.5918	0.4035	0.7405	0.18827
Leaky ReLU	0.8381	0.8373	0.8399	0.00175	0.8522	0.8329	0.8624	0.0193
Tanh	0.8334	0.833	0.8338	**0.00038**	**0.8705**	**0.8661**	0.8735	**0.00433**
Swish	0.8416	0.841	0.8421	**0.0006**	0.8383	0.7969	0.8717	0.04146
GELU	0.8404	0.8397	0.841	0.00066	0.8654	0.855	0.8735	0.01034
Sigmoid	0.8086	0.8079	0.8092	0.00072	0.2982	0.2982	0.2982	0
PMish	0.8391	0.8388	0.8392	**0.00027**	0.8533	0.8089	**0.8763**	0.04441

Figure 18: IGCN Results Part 1

11.4 Swirlu 2

A variation of Swirlu was also tested which is defined as such:

$$\sigma(x) = \left\{ \begin{array}{ll} \alpha x - \frac{1}{\beta} + \frac{\alpha}{\beta} & \text{for } x \geq \frac{1}{\beta} \\ \frac{\alpha x}{e^{|\beta x|-1}} & \text{for } x < \frac{1}{\beta} \end{array} \right\}$$

When $\alpha = 1$ and $\beta = 1$ Swirlu and Swirlu 2 are identical. Swirlu 2 was tested but was omitted from the results as its performance was nearly identical to Swirlu. Meanwhile Swirlu 2 can also become discontinuous which is assumed to be an undesirable trait for an activation function.

	Deep IGCN Citeseer K=2				Deep IGCN PubMed K=2			
Function	Avg	Min	Max	+/-	Avg	Min	Max	+/-
Swirl	0.745	0.7393	0.7491	0.00571	**0.8947**	**0.8926**	**0.8964**	0.00212
Swirl L	0.7452	0.7378	0.7528	0.00759	**0.8935**	**0.8926**	**0.8945**	**0.00099**
Swirlu	**0.7596**	**0.7551**	0.7633	**0.00451**	0.8836	**0.8825**	0.8847	**0.00117**
Swirlu L	**0.7583**	**0.7551**	0.7611	**0.00323**	0.884	0.8815	**0.8872**	0.00317
Swirlax	0.7512	0.7446	0.7543	0.00661	0.8792	0.8638	0.8853	0.01538
Swirlax L	0.751	0.7438	0.7581	0.00721	0.8837	0.8779	0.8865	0.00581
Swirlaxu	0.7554	0.7438	**0.7671**	0.01172	0.8692	0.8456	0.8788	0.02365
Swirlaxu L	**0.7594**	0.7528	**0.7671**	0.00774	0.8748	0.8683	0.8793	0.00649
ReLU	0.5556	0.0714	0.7506	0.48422	0.568	0.2114	0.7045	0.35664
Leaky ReLU	0.7401	0.7077	0.7603	0.03238	0.8757	0.8634	0.8799	0.01222
Tanh	0.7579	**0.7558**	0.7626	**0.00473**	0.8816	0.8803	0.8826	**0.00126**
Swish	0.7458	0.7273	0.7558	0.01848	0.8644	0.8596	0.8675	0.00474
GELU	0.7576	07468	**0.7648**	0.01074	0.8712	0.866	0.8731	0.00519
Sigmoid	0.2328	0.2014	0.3479	0.11503	0.3948	0.3922	0.3965	0.00259
PMish	0.758	0.7476	0.7641	0.01044	0.8716	0.869	0.8731	0.00255

Figure 19: IGCN Results Part 2

References

L. David Aites and David E. Stewart. Supervised learning with informed graph convolution. *IEEE 15th Annual Ubiquitous Computing, Electronics Mobile Communication Conference (UEMCON)*, 2024.

Luis B. Almeida. Multilayer perceptrons. *Handbook of Neural Computation*, 1996.

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2, 2024.

Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 503, 2022.

C. Lee Giles, Kurt D. Bollacker, and Steve Lawrence. Citeseer: An automation citation indexing system. *Proceedings of the third ACM conference on Digital libraries*, 1998.

R. Paul Gorman and Terrence J. Sejnowski. Analysis of hidden units in a layered network trained to classify sonar targets. *Neural Networks*, 1, 1988.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv: Learning*, 2016.

Diederik P Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representation*, 2017.

Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009. URL `https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf`.

Yann LeCun, Bernahrd E. Boser, John S. Denker, and Donnie Hend. Handwritten digit recognition with a back-propagation network. *Neural Information Processing Systems*, 1989.

Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs*, 2, 2010.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521, 2015.

Johannes Lederer. Activation functions in artificial neural networks: A systematic overview. *ArXiv*, abs/2101.09957, 2021.

Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. Dying relu and initialization: Theory and numerical examples. *Communications in Computational Physics*, 28, 2020.

Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. *30th International Conference on Machine Learning*, 2013.

Andrew Kacthites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. Automating the construction of internet portals with machine learning. *Information Retrieval*, 3, 2000.

Prassana Reddy Pulakurthi, Mahsa Mozaffari, Sohail A. Dianat, Jamison Heard, Raghuveer M. Rao, and Majid Rabbani. Enhancing gans with mmd neural architecture search, pmish activation function, and adaptive rank decomposition. *IEEE Access*, 12, 2024.

Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv*, 2018.

Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligeher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 2008.

Zhilin Yang, W. William Cohen, and Ruslan Salakhutdinov. Revisiting semi-supervised learning with graph embeddings. *33rd International Conference on Machine Learning*, 48, 2016.

Zhilu Zhang and Mert R. Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, 2018.